

PRISM — Production-Grade Feature Playbook

Goal: build the strongest credible prototype for TetraTHON's EdTech brief—not a thin demo, not a generic chatbot, and not a false claim of proprietary competitor technology.

Product thesis: PRISM is an adaptive, offline-resilient STEM learning system that turns every learner interaction into explainable evidence, identifies the likely prerequisite gap behind an error or doubt, gives curriculum-grounded remediation, and tells the teacher exactly whom to help and how.

0. The standard: no weak plan

Non-negotiables

1. Every adaptation is explainable. The UI can state the evidence, concept, confidence, and next-step rationale.
2. The core loop works with no LLM and no internet. AI improves tutoring language; it does not replace diagnosis, assessment, progression, or progress records.
3. No static-dashboard theatre. Every learner, attempt, mastery value, misconception cluster, intervention, and graph update derives from persisted event data.
4. No fake ML. We may use trained models only when we can name the dataset, metric, calibration method, and limits. Otherwise use a transparent probabilistic learner model.
5. No black-box root-gap assertion. A diagnosis must include a ranked hypothesis, evidence, confidence, and a clarifying path when uncertain.
6. No cloud-dependent demo. Network loss must preserve current content, attempts, mastery updates, and a deterministic tutor fallback.
7. No hollow AI claim. Every LLM response is grounded in selected curriculum cards, structured learner state, and output validation.

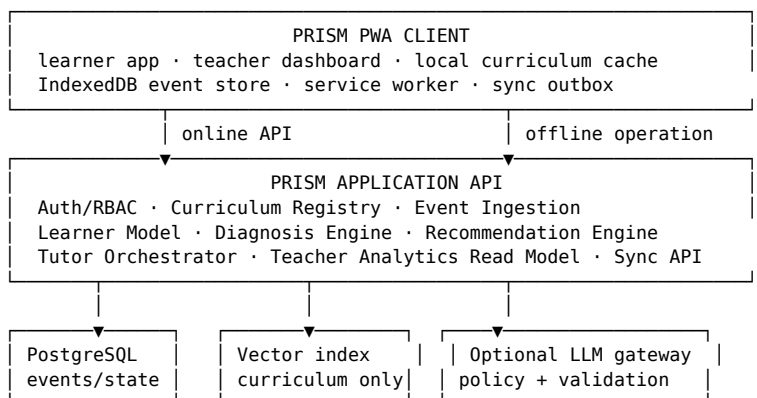
Success experience for a judge

In one 3-minute flow, a judge can see:

- 5-question probe
 - principled level placement
 - a different micro-lesson for that learner
 - a real wrong answer / doubt
 - root prerequisite gap shown on concept graph
 - Socratic repair path with cited content
 - mastery update
 - teacher dashboard identifies the class pattern and suggested intervention
 - airplane-mode continuation and later sync

1. Product architecture

1.1 Architecture at a glance



1.2 Logical services

Service • Responsibility • Must function without LLM?

Curriculum Registry • concepts, prerequisites, questions, rubrics, hints, citations • Yes

Assessment Service • score answers, classify misconception/error • Yes

Learner Model • update concept mastery and uncertainty from events • Yes

Diagnosis Engine • infer likely root gap through concept graph • Yes

Recommendation Engine • choose next micro-step and intervention • Yes

Tutor Orchestrator • convert verified state into Socratic/direct language • Fallback required

Analytics Read Model • classroom aggregation and teacher actions • Yes

Sync Service • idempotent event upload and reconciliation • Queues offline

1.3 Competitor-derived lessons—applied honestly

Source • Public pattern • PRISM application

Duolingo / Birdbrain • individual responses feed learner/concept estimates and exercise selection • event-level signals + adaptive recommendation engine

Khan Academy / Khanmigo • structured learner context, curriculum grounding, Socratic tutoring and safety iteration • bounded tutor contract; context is state, not raw chat only

Oppia • authored answer classification and explicit skill progress • deterministic rubric/misconception rules before LLM explanation

Kolibri • offline-first learning logs and syncable state • local event store + eventual sync

Open edX • persisted learner × learning-object state • durable progress records and dashboard read models

pyKT research • ordered concept/question/response traces • future-ready event schema; calibrated knowledge tracing later

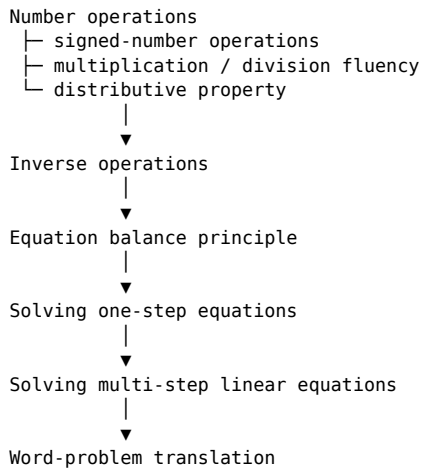
Boundary: PRISM does not claim any vendor’s private model, prompt, thresholds, infrastructure, or data.

2. Curriculum Intelligence Layer

2.1 Build a real concept graph, not decorative nodes

Initial scope

Class 8 Mathematics — Linear Equations in One Variable



Concept contract

```

{
  "id": "eq.inverse_operations",
  "label": "Inverse operations",
  "grade": 8,
  "prerequisites": ["num.signed_operations", "num.mul_div_fluency"],
  "learning_objective": "Use the inverse operation to isolate a variable.",
  "mastery_criterion": {
    "min_probability": 0.78,
    "min_independent_correct": 3,
    "max_recent_error_streak": 0
  },
  "misconceptions": ["eq.divide_only_constant", "eq.sign_not_transferred"],
  "citation": {
    "source": "NCERT-aligned authored lesson card",
    "locator": "LE-02"
  }
}
  
```

Edge contract

```

{
  "from": "num.signed_operations",
  "to": "eq.inverse_operations",
  "relationship": "prerequisite",
  "strength": 0.85,
  "rationale": "Sign errors prevent valid inverse-operation application."
}
  
```

2.2 Question and misconception authoring

Every assessment item must be rich enough to support grading, remediation, and analytics.

```

{
  "id": "q.eq.03",
  "concept_id": "eq.inverse_operations",
  "difficulty": 0.52,
  "prompt": "Solve:  $3x - 5 = 16$ ",
  "answer_type": "numeric",
  "expected_answer": "7",
  "solution_steps": ["Add 5 to both sides", "Divide both sides by 3"],
  "rubric": [
    {"pattern": "11", "error_tag": "eq.stops_before_division", "confidence": 1.0},
    {"pattern": "-7", "error_tag": "eq.sign_not_transferred", "confidence": 1.0}
  ],
  "hint_ladder": [
    "What must happen to both sides before x can be isolated?",
    "First undo subtracting 5. What equation do you get?",
    "Now 3 is multiplying x. What is the inverse operation?"
  ]
}
  
```

```

    "feedback": {
      "correct": "Correct: you used inverse operations in the right order.",
      "eq.stops_before_division": "You correctly removed 5, but x is still multiplied by
3."
    }
  }
}

```

Authoring quality gate

An item cannot enter production unless it has:

- concept and prerequisite tags;
- correct solution and independent scoring rule;
- at least one anticipated misconception or explicit “unknown error” fallback;
- hint ladder and a direct explanation;
- citation/lesson-card reference;
- accessibility text and mobile-friendly rendering;
- unit test fixtures for correct, common wrong, malformed, and empty responses.

3. Learner Model: probabilistic, explainable, and calibratable

3.1 Why not use a simplistic score

A raw percentage cannot distinguish:

- one lucky correct answer from stable mastery;
- a correct answer after three hints from independent success;
- a new error from a recurring prerequisite gap;
- uncertainty caused by insufficient evidence.

PRISM tracks mastery probability plus uncertainty per concept.

3.2 Prototype-grade Bayesian Knowledge Tracing (BKT)

Use BKT as the primary explainable model. It is stronger than score averaging and realistic without a massive private dataset.

For each learner-concept pair maintain:

$P(L)$	probability the learner currently knows the concept
$P(T)$	probability of learning between opportunities
$P(S)$	probability of a slip despite knowing
$P(G)$	probability of a lucky guess despite not knowing
n	evidence count
	confidence interval / uncertainty proxy

Correct response update

```
posterior_known =
  P(L)*(1-P(S)) / [P(L)*(1-P(S)) + (1-P(L))*P(G)]

P(L)_next = posterior_known + (1-posterior_known)*P(T)
```

Incorrect response update

```
posterior_known =
  P(L)*P(S) / [P(L)*P(S) + (1-P(L))*(1-P(G))]
```

$P(L)_{next} = posterior_known + (1-posterior_known)*P(T)$

Starting parameters—clearly prototype assumptions

```
P(L)_initial = 0.35
P(T) = 0.12
P(S) = 0.08
P(G) = 0.20
```

Adjust by evidence class:

```
hint used      → increase P(G) and reduce mastery gain
first attempt  → normal update
repeated error → lower confidence / route to diagnosis
very fast guess → mark lower evidence weight
self-explanation accepted → modest learning transition bonus
```

These are documented assumptions, not an assertion that they are empirically calibrated. For a strong prototype, calibrate them against a held-out authored simulation dataset and report the limitation.

3.3 Mastery state

```
{
  "learner_id": "student-07",
  "concept_id": "eq.inverse_operations",
  "p_know": 0.62,
  "evidence_count": 4,
  "independent_correct_count": 1,
  "recent_error_tags": ["eq.stops_before_division"],
  "uncertainty": "medium",
  "updated_at": "2026-07-17T09:00:00Z"
}
```

3.4 Mastery presentation

Never show a falsely precise percentage as unquestionable truth.

Model state • Learner wording • Teacher wording

$p_know < 0.40$ • “Let’s rebuild this idea.” • “Needs prerequisite repair.”

$0.40-0.70$ • “You are getting there—one more check.” • “Developing; evidence still limited.”

≥ 0.70 + independent evidence • “Ready for the next challenge.” • “Likely mastered; confirm with transfer item.”

4. Adaptive Sequencing Engine

4.1 Two-stage decision: eligibility then ranking

Stage A — eligibility

A learning item is eligible only when:

- it belongs to the learner's active path OR is needed prerequisite repair;
- its prerequisite readiness meets the threshold;
- it has not exceeded exposure/repetition limits;
- it is locally cached when offline;
- it is not blocked by a currently unresolved severe misconception.

Stage B — ranking

```
need(c)           = 1 - P_know(c)
prereq_ready(c)   = weighted mean mastery of prerequisite nodes
challenge_fit(i)  = 1 - abs(item_difficulty(i) - target_challenge)
info_gain(i)      = expected reduction in uncertainty
spacing(i)        = time since last relevant exposure
misconception_bonus = 1 if item tests current suspected gap else 0
fatigue_penalty   = grows with session length and repeated failure

priority(i) =
  0.24*need
+ 0.20*prereq_ready
+ 0.18*challenge_fit
+ 0.16*info_gain
+ 0.10*spacing
+ 0.08*misconception_bonus
- 0.04*fatigue_penalty
```

Select the highest-ranked item, then store the reason trace.

4.2 Explainability trace

```
{
  "recommendation": "repair.eq.inverse_operations.card-1",
  "reason": [
    "Recent independent response indicates a likely inverse-operations gap.",
    "This is a direct prerequisite of the current target: multi-step equations.",
    "The learner has not yet demonstrated independent success on this concept."
  ],
  "model_inputs": {
    "p_know": 0.31,
    "current_target": "eq.multi_step",
    "prerequisite_strength": 0.90,
    "diagnosis_confidence": 0.78
  }
}
```

This is the element competitors' public product pages rarely make visible and judges will remember.

4.3 The diagnostic probe

Do not make all five questions measure the same difficulty.

Question • Primary concept • Goal

- | |
|---|
| 1 • signed operations • identify foundational blocker |
| 2 • inverse operation • verify direct prerequisite |
| 3 • one-step equation • grade-level core capability |

4 • multi-step equation • transfer / procedure control

5 • word translation • advanced application / intent representation

Use the learner model after the probe, not only total marks. A learner with 3/5 but a direct prerequisite failure should not be pushed into the same path as one who misses only advanced transfer.

5. Root-Gap Diagnosis Engine

5.1 The product differentiator

A generic tutor answers the visible question. PRISM should identify the most likely prerequisite failure causing the visible question.

```
visible error or doubt
→ current target concept
→ deterministic answer/error classification
→ evidence retrieval from recent attempts
→ prerequisite-graph traversal
→ ranked root-gap hypotheses
→ confirm/repair/return loop
```

5.2 Hybrid diagnosis pipeline

Step 1: classify structured evidence first

Priority order:

1. exact numeric/symbolic rubric;
2. step-level parser or expression equivalence checker;
3. authored regex/semantic patterns for common misconceptions;
4. optional constrained LLM semantic assessor;
5. unknown rather than a fabricated classification.

Step 2: graph search

Search target concept ancestors up to depth 2 initially. Prevent “everything is a root gap” by applying:

- max 5 candidates
- depth penalty
- no diagnosis from a single weak/ambiguous chat sentence
- evidence freshness decay

Step 3: rank hypotheses

```
score(gap) =
  0.38 * direct_error_evidence
+ 0.22 * prerequisite_failure_history
+ 0.16 * graph_proximity
+ 0.12 * recurrence
+ 0.07 * time/hesitation_evidence
+ 0.05 * bounded_semantic-assessor_confidence
```

Step 4: confidence and action policy

```
if score_1 >= 0.72 and score_1 - score_2 >= 0.18:  
    action = "guided remediation"  
elif score_1 >= 0.50:  
    action = "ask discriminating question"  
else:  
    action = "state uncertainty, clarify learner intent"
```

5.3 Discriminating questions

Each misconception maps to one very short test that separates competing root gaps.

Hypothesis A: cannot use inverse operations
Hypothesis B: cannot divide coefficients

Question: "After $3x = 21$, what operation isolates x ?"

answers "subtract 3" → inverse-operation gap
answers "divide by 3" but computes wrong → arithmetic/division gap

This is much stronger than confidently hallucinating a diagnosis.

5.4 Graph visualization

Visual state per node:

- mastered — green; sufficient independent evidence
- developing — amber; probability/uncertainty present
- at_risk — red; repeated evidence of misconception
- suspected_root_gap — red outline/pulse; diagnosis hypothesis
- recommended_next — blue highlight
- not_assessed — neutral

On node click, expose the evidence ledger—not an opaque score.

6. AI Tutor: grounded, policy-driven, and multimodal

6.1 Tutor responsibilities

The tutor may:

- clarify the learner's doubt in context;
- ask a Socratic question;
- give one hint at a time;
- give a short direct explanation after policy conditions;
- link to a local concept card and citation;

- create a short practice check;
- report uncertainty and request clarification.

The tutor may not:

- modify mastery directly;
- assert a root gap without a diagnosis record;
- invent a citation or lesson rule;
- rely on unrestricted internet search for curriculum truth;
- reveal policy, hidden prompt, private student data, or model internals.

6.2 Retrieval architecture

```
request
→ load active concept + direct prerequisites + authored lessons
→ filter to grade/subject/scope
→ retrieve top curriculum chunks from local/vector index
→ provide only cited chunks to tutor
→ generate structured response
→ validate citations, mode, and safety policy
→ render response and practice check
```

Important: for one-topic hackathon scope, curated structured context is superior to pretending a large RAG corpus is needed. Use retrieval only when the corpus is broad enough to justify it.

6.3 Tutor response schema

Force the model to return structured output. Text alone is not an application contract.

```
{
  "response_mode": "socratic_hint",
  "message": "What should happen to both sides before x is alone?",
  "concept_ids": ["eq.inverse_operations"],
  "citation_ids": ["LE-02"],
  "confidence": "medium",
  "next_action": "await_learner_attempt",
  "safety_flags": []
}
```

Reject/retry output if:

- no known citation supports a factual explanation;
- the response gives a final answer while policy requires a hint;
- it references an unknown concept;
- it claims high diagnostic certainty where diagnosis is low-confidence;
- JSON/schema validation fails.

6.4 Socratic-to-direct policy

```
Attempt 0: ask a focused thinking question
Attempt 1 wrong: give a conceptual hint
Attempt 2 wrong: show the relevant operation / visual reasoning
Attempt 3 wrong or learner requests "show me": direct worked explanation
After explanation: one isomorphic transfer question
```

The final transfer question is mandatory: explanation without new evidence is not learning proof.

6.5 Handwritten-doubt image pipeline

This is valuable, but must be a fully tested feature or be clearly marked optional.

```
image
  → quality check (blur, crop, handwriting confidence)
  → OCR / math expression extraction
  → present extracted text to learner for confirmation
  → same diagnosis pipeline as text
```

Never silently treat low-confidence OCR as fact. If extraction is uncertain, ask the learner to confirm/edit the transcription.

7. Teacher Intelligence Dashboard

7.1 Design principle

Do not make the teacher read chat logs. Make the dashboard answer:

Which students need which intervention, based on what evidence, right now?

7.2 Teacher dashboard panels

A. Cohort command center

- learner count by Foundational / Grade-Level / Advanced band;
- active/completed/offline-pending sessions;
- concept heatmap: mastery probability + confidence + error rate;
- top misconception clusters;
- “intervene now” ranked recommendations.

B. Student intervention card

```
Student: Anya
Current path: Grade-Level
Current target: Multi-step equations
Likely blocker: Inverse operations (0.78 confidence)
Evidence: 3 recent attempts stopped after isolating the constant
Recommended action: 2-minute inverse-operation mini-whiteboard check
Status: 2 local attempts pending sync
```

C. Misconception cluster view

```
Cluster: Stops before dividing the coefficient
Affected: 9 / 24 learners
Trend: +18% today
Current concept: inverse operations
Suggested intervention:
  “Ask the class: in  $3x = 21$ , is  $x$  alone yet? What operation reverses  $\times 3$ ?”
```

7.3 Cohort algorithm

```
impact(cluster) =  
  0.42 * affected_distinct_learners / active_learners  
+ 0.22 * recent_incorrect_rate  
+ 0.18 * repeat_error_rate  
+ 0.10 * downstream_dependency_risk  
+ 0.08 * trend_growth
```

Fairness guardrail

Do not classify a learner as “weak.” Use evidence-aware states:

- insufficient evidence;
- needs prerequisite support;
- developing;
- ready for extension.

Never rank/label students publicly. Teacher views must be role-protected.

7.4 Teacher action loop

```
cluster detected  
  → teacher sees evidence-backed recommendation  
  → teacher assigns targeted repair activity / group  
  → learner events resume  
  → dashboard reports post-intervention change
```

This closes the loop missing from many generic dashboards.

8. Offline-First and Reliability Architecture

8.1 Offline is a feature, not a fallback error message

Cached package

- current + next micro-lesson path
- question bank and answer rubrics
- hint ladders / direct explanations
- concept graph
- visual assets and accessibility labels
- learner’s latest known mastery state

Local-first data model

```
{  
  "event_id": "uuid",  
  "device_id": "uuid",  
  "sequence": 42,  
  "event_type": "attempt_recorded",  
  "payload": {"...": "..."},  
  "created_at_device": "timestamp",  
  "sync_status": "pending"
```

}

8.2 Sync protocol

1. Client appends event to IndexedDB transactionally.
2. UI updates from local projection immediately.
3. Sync worker posts ordered unacknowledged events with idempotency keys.
4. Server stores unseen events; duplicate event IDs are no-ops.
5. Server returns accepted IDs + canonical derived mastery snapshot.
6. Client marks accepted events synced and reconciles projection.

8.3 Conflict policy

- Attempts are immutable append-only facts.
- Server never overwrites an event based solely on a client clock.
- Mastery is a derived projection; it can be recalculated after merged events.
- Show sync state openly. Do not hide stale dashboards behind a “live” badge.

8.4 Reliability tests

Must demonstrate:

- complete a micro-lesson in airplane mode;
- close/reopen app and resume same state;
- reconnect, sync exactly once, and show no duplicate attempt;
- LLM timeout returns authored fallback in under a defined UX state;
- backend outage leaves queued learning events intact.

9. Privacy, safety, and trust

9.1 Data minimization

Store only data necessary for adaptive learning:

- pseudonymous learner ID;
- learning events, mastery state, session status;
- consented uploaded doubt image, with deletion route;
- no unnecessary contact data, biometric data, or unrestricted chat analytics.

9.2 Role-based access control

Role • Access

Learner • own lessons, attempts, graph, tutor history

Teacher • assigned cohort summaries and permitted student evidence

Every API query enforces server-side classroom membership. Hiding a frontend button is not authorization.

9.3 AI safety controls

- curriculum-scoped context and citations;
- output schema validation;
- prompt injection resistance: learner content is data, never instruction;
- moderated response / refusal path for unsafe input;
- no final-answer shortcut where the tutor policy requires guidance;
- audit log for tutor policy mode, source IDs, and fallback use;
- “report response” feedback control.

10. Evaluation: prove the prototype, do not just show it

10.1 Functional acceptance suite

Requirement • Pass condition

Diagnostic • 5 answer patterns deterministically assign expected band/path

Microlearning • every path has a 10-minute flow, 3 checks, feedback, completion event

Root gap • common misconception maps to correct candidate and shows evidence trace

Uncertainty • ambiguous doubt triggers a discriminating question—not confident claim

Tutor • cited Socratic response validates against response schema

Dashboard • synthetic persisted events change cohort clusters and intervention ranking

Offline • local attempts survive reload and sync idempotently

Accessibility • keyboard path, readable focus state, semantic labels, mobile layout

10.2 Diagnostic/learner-model evaluation

Create an authored scenario set before demo:

- foundational learner: signed-number blocker
- grade-level learner: inverse-operation omission
- advanced learner: succeeds procedure, fails word translation
- ambiguous learner: conflicting clues; system must ask a discriminating question
- recovery learner: remediation then independent transfer success

Measure:

band classification accuracy on authored cases
root-gap top-1 / top-3 match against authored ground truth
unsafe-overconfidence rate
fallback completion rate without LLM/network
recommendation reason completeness
sync duplicate-event rate

10.3 LLM evaluation

Use a fixed adversarial and pedagogical test set:

- asks for answer without work;
- prompt-injection attempt inside doubt text;
- irrelevant request;
- wrong root-gap assumption;
- unsupported factual question;
- serious/unsafe content;
- citation request;
- empty/garbled OCR input.

Score:

```
schema-valid rate
citation-valid rate
Socratic-policy compliance
unsupported-claim rate
fallback success rate
median response latency
```

No AI feature is “done” because it sounds fluent.

11. Build order: strongest path, not shortcut path

Phase 1 — Foundation

1. curriculum graph and item-authoring format;
2. relational schema + append-only event model;
3. deterministic answer grading and misconception tagging;
4. BKT learner model plus test fixtures;
5. diagnostic classification and adaptive sequencing trace.

Gate: a scripted learner simulation produces expected state transitions and recommendations.

Phase 2 — Complete learning loop

1. learner PWA flows;
2. all three differentiated micro-lessons;
3. concept graph visualization;
4. root-gap engine and discriminating-question flow;
5. teacher dashboard projections.

Gate: every stated hackathon requirement works end-to-end without an LLM.

Phase 3 — Reliability + offline

1. service worker/cache strategy;
2. IndexedDB event store and outbox;
3. sync API/idempotency;
4. reconnect/reconciliation UX;
5. fault injection tests.

Gate: airplane-mode demo survives close/reopen and one reconnect.

Phase 4 — AI enhancement

1. grounded tutor context builder;
2. structured-output schema and validator;
3. Socratic/direct policy state machine;
4. semantic assessor only for unstructured doubts;
5. fallback and safety tests.

Gate: tutor adds quality but never owns the core assessment/diagnosis decision.

Phase 5 — Submission quality

1. fresh-install runbook and seed scenarios;
2. test run evidence;
3. 2-minute and 5-minute demo scripts;
4. source-backed architecture slides;
5. limitation statement and future data/validation roadmap.

12. Definition of “best possible” for this hackathon

PRISM wins credibility if it demonstrates all of the following simultaneously:

- Adaptive: different valid learner evidence produces different paths.
- Diagnostic: it identifies a prerequisite hypothesis, not only a wrong answer.
- Explainable: the learner and teacher see the evidence behind an adaptation.
- Pedagogical: the tutor guides, verifies transfer, and cites its curriculum.
- Teacher-useful: the dashboard recommends a next intervention.
- Offline-resilient: learning continues and syncs truthfully.

- Technically honest: deterministic core, calibrated claims, public-source provenance, explicit limitations.

A polished generic chat app cannot meet this bar. A real learner model, concept graph, event spine, root-gap engine, and offline workflow can.

Public-source baseline

- Duolingo Birdbrain: <https://blog.duolingo.com/learning-how-to-help-you-learn-introducing-birdbrain/>
- Duolingo frontend prediction: <https://blog.duolingo.com/frontend-prediction/>
- Khan Academy prompt-engineering methodology:
<https://blog.khanacademy.org/khan-academys-7-step-approach-to-prompt-engineering-for-khanmigo/>
- Khan Academy teacher reporting: <https://support.khanacademy.org/hc/en-us/articles/360031129891-What-reporting-options-are-available-on-Khan-Academy-for-teachers-to-track-student-performance>
- Oppia: <https://github.com/oppia/oppia>
- Kolibri: <https://github.com/learningequality/kolibri>
- Open edX: <https://github.com/openedx/openedx-platform>
- Adapt Framework: https://github.com/adaptlearning/adapt_framework
- pyKT Toolkit: <https://github.com/pykt-team/pykt-toolkit>